

Relatório técnico: acoplamento MFSim–RTS e caminhos de paralelização MPI

Ênfase em decomposição de domínio e nos métodos FAM e DOM

Análise baseada no código fornecido (`src_term/RTS`)

10 de fevereiro de 2026



Figura: logotipo/arte do RTS incluída no repositório analisado.

Escopo. Este documento descreve (i) como o RTS (Radiative Transfer Simulator) foi acoplado ao MFSim, (ii) como os métodos FAM (Finite Angle Method) e DOM (Discrete Ordinates Method) são executados no RTS e quais estruturas de dados eles usam, e (iii) quais estratégias de paralelização com MPI são mais promissoras para o RTS, com prós e contras, com foco em decomposição de domínio.

Sumário

1	Visão geral do acoplamento MFSim–RTS	2
1.1	Campos trocados entre os códigos	2
1.2	Malhas: AMR (MFSim) vs. cartesiana uniforme (RTS)	2
2	Estrutura do código e fluxo de chamadas	2
2.1	Principais módulos e responsabilidades	2
2.2	Fluxo completo do acoplamento e pontos MPI	3
3	Interpolação e mapeamento geométrico entre as malhas	3
3.1	Estrutura <code>interp_map</code> e a ideia do “dono” do ponto	3
3.2	Como o mapa é construído e por que isso pode custar caro	4
3.3	Interpolação MFSim $\rightarrow$ RTS	4
3.4	Interpolação RTS $\rightarrow$ MFSim	5
4	RTS: organização do solver e papel dos métodos FAM e DOM	5
4.1	Seleção do método e ponto de entrada	5
4.2	DOM (Discrete Ordinates Method)	5
4.3	FAM (Finite Angle Method)	6

5	Paralelização atual: o que existe hoje e limitações	7
5.1	O que está paralelizado de fato	7
5.2	Principais gargalos observáveis no desenho atual	7
6	Estratégias de paralelização MPI para o RTS (com prós e contras)	7
6.1	Opção 0: reduzir consumo de memória sem paralelizar o solver	7
6.2	Opção 1: decomposição por bandas espectrais (MPI sobre IBND)	8
6.3	Opção 2: decomposição angular (MPI sobre direções/ângulos)	8
6.4	Opção 3: decomposição de domínio espacial (MPI sobre i, j, k)	9
6.5	Opção 4: híbrido (domínio espacial + ângulos + bandas)	10
6.6	Opção 5: “decomposição de domínio com fontes defasadas” (Jacobi) para evitar wavefront	10
6.7	Opção 6: reinterpretação do FAM como “feixes/rays” e paralelizar por caminhos	11
7	Recomendação prática: caminho incremental	11
8	Checklist de alterações no código para uma versão RTS paralela por domínio	11
8.1	1) Definir subdomínio RTS por processo	12
8.2	2) Refatorar arrays globais do RTS para arrays locais	12
8.3	3) Implementar comunicação de intensidades nos sweeps	12
8.4	4) Resíduo e convergência global	12
8.5	5) Acoplamento com MFSim sem operações globais	12
9	Conclusões	12

1 Visão geral do acoplamento MFSim-RTS

O acoplamento MFSim-RTS no código analisado é implementado como um *módulo de fonte* (termo radiativo) dentro do MFSim. A chamada principal é feita por `rad_trigger` (`src_term/RTS/rad_core.f90`), que decide quando inicializar o RTS e quando resolver a equação de transporte radiativo (RTE).

1.1 Campos trocados entre os códigos

De forma simplificada, o MFSim fornece ao RTS campos volumétricos necessários para a radiação e recebe de volta os termos radiativos:

- **MFSim → RTS:**  *Em volume de controle*
 - Temperatura do escoamento: `energy_temp` (MFSim) → `T_energy` (RTS).
 - (Opcional, para propriedades radiativas de gás) Pressão e espécies (ex.: CO₂ e H₂O): `ns_pres, react_var(:, species)` (MFSim) → `P_species, Y_species(:, :, :, 1:2)` (RTS).
- **RTS → MFSim:**
 - Incidência radiativa: `G` (RTS) → `G_rad` (MFSim).
 - Fonte volumétrica radiativa (divergência do fluxo radiativo): `S_rad` (RTS) → `Q_rad` (MFSim).
 - Fluxos radiativos nas paredes: `Q_radw` (RTS) → `Q_radww_x, Q_radww_y, Q_radww_z` (MFSim, por faces).

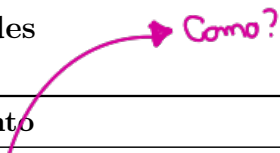
1.2 Malhas: AMR (MFSim) vs. cartesiana uniforme (RTS)

O MFSim trabalha com malha adaptativa em blocos/patches (AMR), acessada via `headlev` (lista ligada por nível; cada nó contém `grid%ix, iy, iz, mx, my, mz` e espaçamentos `hx, hy, hz`). Já o RTS constrói uma malha cartesiana uniforme própria com dimensões definidas por `rad_nx, rad_ny, rad_nz` (MFSim) e gera tamanhos estendidos `nxi=nx+2`, etc. (RTS), com pontos adicionais típicos de células fantasma.

Consequentemente, o acoplamento depende criticamente de **interpolação entre malhas** e de **decisões de paralelização** para evitar gargalos na transferência de dados. *Preciso que desente 😊*

2 Estrutura do código e fluxo de chamadas

2.1 Principais módulos e responsabilidades



Arquivo/módulo	Papel no acoplamento
<code>rad_core.f90</code> (module <code>rad_core</code>)	Entrada principal do termo radiativo no MFSim. Implementa <code>rad_trigger</code> , chama <code>start_connection</code> (inicialização do RTS), faz interpolação MFSim→RTS , chama <code>rtesolve</code> (RTS), faz broadcast dos resultados e interpola RTS→MFSim .
<code>RTS_connection.f90</code> (module <code>RTS_connection</code>)	Inicializa e aloca variáveis do RTS no contexto do MFSim. Lê <code>input.rts</code> , define dimensões, constrói malha do RTS (mesh), aloca arrays e chama <code>rad_start</code> e <code>initialization</code> .
<code>rad_interface.f90</code> (module <code>rad_interface</code>)	Faz o “encaixe” geométrico e a interpolação entre MFSim e RTS. Mantém um <code>interp_map</code> que mapeia cada ponto da malha do RTS para um patch do MFSim (ponteiros) e indica em qual processo MPI aquele ponto deve ser computado.

O que é esse patch :-)

`sources/RTS_radiation.f90` implementa o solver radiativo do RTS. A rotina `rtesolve` escolhe entre P1, DOM ou FAM, e cada método resolve a RTE com iterações até `rad_tol` ou `ITMAX`.

2.2 Fluxo completo do acoplamento e pontos MPI

O fluxo atual (como está no código) pode ser descrito pelo diagrama da Figura 1. A característica mais marcante é que o **solver RTS roda apenas no processo `proc_id==0`**, enquanto os demais processos ajudam a construir os campos de entrada via interpolação e depois recebem os resultados via broadcast.

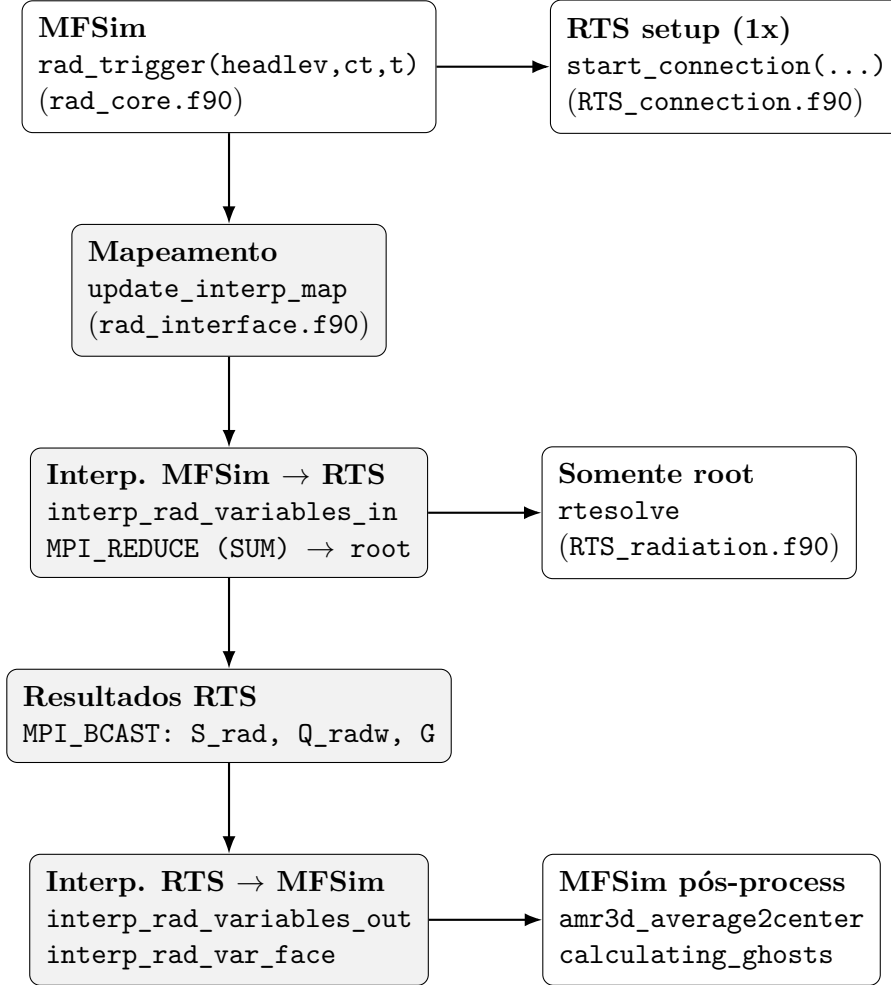


Figura 1: Fluxo atual do acoplamento MFSim–RTS. O uso de `MPI_REDUCE` agrega a malha radiativa no *root*, que executa o RTS, e depois dissemina os campos via `MPI_BCAST`.

3 Interpolação e mapeamento geométrico entre as malhas

3.1 Estrutura `interp_map` e a ideia do “dono” do ponto

Em `rad_interface.f90` existe um array 3D `interp_map(i,j,k)`. Cada elemento armazena, entre outros, os seguintes conceitos:

- **in_proc**: indica se o ponto $(x_c(i), y_c(j), z_c(k))$ pertence ao subdomínio (caixa física) do processo MPI atual. Essa decisão é feita por `is_in_proc(xc,yc,zc)` em `src_amr/eul_coords.f08`, usando limites `a1..b1`, `a2..b2`, `a3..b3`.

- `is_edge`: marca pontos que estão sobre arestas/cantos do domínio do RTS (`is_RTS_domain_edge`). O código atual *zera* esses pontos na interpolação de entrada.
- `headlev_ptr` e `level_cell`: um ponteiro para o patch do MFSim que contém o ponto e o nível AMR correspondente, preenchidos quando `is_in_patch(patch, ...)` retorna verdadeiro.

A intenção é garantir uma propriedade essencial para o uso de `MPI_REDUCE(SUM)` como “gather”:

Cada ponto do RTS deve ser interpolado por exatamente um processo MPI (e ficar zero em todos os outros).

Se dois processos considerarem o mesmo ponto como “seu” (por comparações numéricas, bordas inclusivas, etc.), o `MPI_REDUCE` somará duas contribuições e o valor ficará incorreto. O tratamento de `is_edge` e as condições de borda em `is_in_proc` são tentativas de evitar essa duplicidade.

3.2 Como o mapa é construído e por que isso pode custar caro

O mapa é alocado uma vez em `start_interp_map(...)` e atualizado em `update_interp_map(...)`. O padrão atual é:

1. Para cada nível AMR `l=1..ltop`:
2. Percorrer a lista ligada de patches `curr => headlev(l)%next`.
3. Para *cada* patch, varrer *toda* o grid do RTS `i=1..nxi`, `j=1..nyi`, `k=1..nzi` e checar se o ponto está dentro do patch.

Esse desenho implica custo assintótico do tipo

$$\mathcal{O}(N_{\text{patches}} N_{\text{pontos RTS}})$$

por atualização do mapa, o que pode ser proibitivo se o AMR gerar muitos patches e a malha radiativa for fina.

Sugestão imediata (sem mudar o modelo de paralelismo). Ao invés de varrer toda a malha radiativa para cada patch, é possível:

- computar **intervalos de índices** ($i_{\min}, i_{\max}$) do RTS que interceptam a caixa do patch (por aritmética de índices na malha uniforme), e só então varrer aquele bloco; e/ou
- inverter o laço: para cada ponto do RTS, buscar o patch que o contém usando uma estrutura de aceleração (lista de caixas + busca, árvore, hash espacial).

Isso reduz significativamente comparações e melhora escalabilidade mesmo antes de paralelizar o solver.

3.3 Interpolação MFSim $\rightarrow$ RTS

A interpolação em si é feita em `vrlb_interp_rad_mesh(...)` (chamada por `interp_rad_variables_in`):

- cada processo varre *toda* a malha do RTS, mas pula pontos onde `interp_map%in_proc` é falso;
- para pontos ativos, usa o ponteiro do patch e chama `point_interpolation(...)` (interpolação linear/cúbica no MFSim).
- o campo parcial `rts_var_0` é reduzido por soma no root:

Listing 1: Esquema do acoplamento atual para MFSim $\rightarrow$ RTS.

```
rts_var = 0
rts_var_0 = 0
call vrlb_interp_rad_mesh(headlev, rts_var_0, mfsim_var, ...)
call MPI_REDUCE(rts_var_0, rts_var, rad_iug, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD,
               ierr)
```

3.4 Interpolação RTS $\rightarrow$ MFSim

Após o RTS resolver a RTE no root, três campos são distribuídos para todos:

Listing 2: Broadcast dos resultados radiativos antes da interpolação de volta.

```
call MPI_BCAST(S_rad, rad_iug, MPI_DOUBLE, 0, MPI_COMM_WORLD, ierr)
call MPI_BCAST(Q_radw, rad_iug, MPI_DOUBLE, 0, MPI_COMM_WORLD, ierr)
call MPI_BCAST(G, rad_iug, MPI_DOUBLE, 0, MPI_COMM_WORLD, ierr)
```

Em seguida cada processo interpola:

- S_rad para Q_rad (campo volumétrico),
- G para G_rad (campo volumétrico),
- Q_radw para fluxos em faces em MFSim (interp_rad_var_face), preenchendo apenas as células nas fronteiras físicas ($i=1$, $i=nx$, etc.).

4 RTS: organização do solver e papel dos métodos FAM e DOM

4.1 Seleção do método e ponto de entrada

No RTS, a rotina rtesolve (sources/RTS_radiation.f90) escolhe o modelo por model_stc:

- model_stc=1: método P1
- model_stc=2: método DOM
- model_stc=3: método FAM

Esse inteiro é definido a partir da string rad_model lida de input.rts (em sources/RTS_input.f90).

4.2 DOM (Discrete Ordinates Method)

Discretização angular e estrutura de dados. No DOM, a intensidade radiativa é armazenada como

$$I \equiv IG(i, j, k, l, m),$$

onde (i, j, k) indexa a malha espacial do RTS, $l = 1..nq$ indexa direções de quadratura por octante, e $m = 1..8$ indexa os oito octantes (combinações de sinais de (μ, η, ξ)).

Ciclo iterativo. Na forma cinza (DOM_gray), o ciclo por iteração faz:

1. DOMbound_in: calcula intensidade de entrada nos contornos, com reflexão difusa baseada em emissividade.
2. RHS_SM_DOM: monta o termo fonte angular $S_m(i, j, k, l, m)$, incluindo emissão e espalhamento.
3. orthogonal_loop: varre a malha em *sweeps* por octante e atualiza IG.
4. DOMbound_out: atualiza as intensidades de saída nos contornos.
5. avalia resíduo res_rad e decide convergência.

Sweep por octantes. O `orthogonal_loop` percorre a malha em diferentes ordens de (i, j, k) dependendo do octante. Por exemplo, no octante 1 (propagação $+x, +y, +z$) a varredura é em ordem crescente:

$$k = 2..nzp, \quad j = 2..nyp, \quad i = 2..nxp.$$

Para cada célula, a intensidade usa valores a montante: $IG(i-1, \dots), IG(\dots, j-1, \dots), IG(\dots, k-1, \dots)$.

Cálculo dos resultados. Depois de convergir, calcula-se a incidência:

$$G(i, j, k) = \text{int}_{4\pi} I, d\Omega \text{approx} \sum_{m=1}^8 \sum_{l=1}^{nq} I(i, j, k, l, m), W_q(l),$$

e então

$$S_{rad} = kappaleft(4\sigma T^4 - G_{right}),$$

além dos fluxos nas paredes via `wall_fluxes_DOM`.

4.3 FAM (Finite Angle Method)

Discretização angular. No FAM, o espaço angular é discretizado em:

- **nt:** número de ângulos em θ (polar),
- **np:** número de ângulos em ϕ (azimutal).

A intensidade radiativa é $IG(i, j, k, l, m)$ com $l = 1..nt$ e $m = 1..np$.

Ciclo iterativo e sweeps. O ciclo (`FAM_gray` e `FAM_nongray`) tem a mesma estrutura conceitual do DOM:

1. **FAMbound_in:** impõe condições de entrada nos contornos, com reflexão em superfícies.
2. **RHS_SM_FAM:** monta o termo fonte angular. Com espalhamento, há um duplo somatório em todos os ângulos (ell, m) para cada direção (l, m).
3. **angular_loop:** faz sweeps em oito octantes, mas particiona índices de ($theta, phi$) em faixas (P2, P3, P4, T2) para separar sinais de cossenos diretores.
4. **FAMbound_out:** calcula intensidades de saída.

Custo do espalhamento no FAM. Em `RHS_SM_FAM`, o termo de espalhamento usa um somatório sobre todos os ângulos discretos:

$$\text{textSMSUM} = \sum_{ell=1}^{nt} \sum_{m=1}^{np} I(i, j, k, ell, m); \Phi(ell, m, l, m); d\Omega_{ell, m}.$$

Isso faz o custo do espalhamento crescer como

$$O(N_{cells})$$

, (nt

, np)²) (quando espalhamento ativo), o que costuma ser o *hotspot* do método.

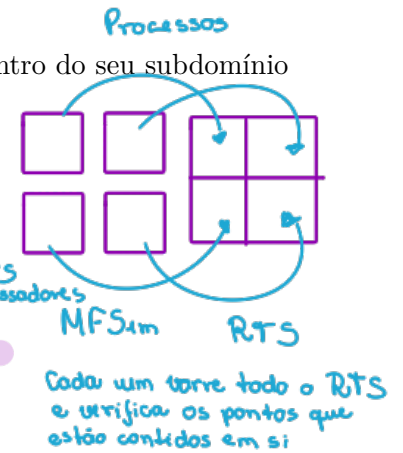
5 Paralelização atual: o que existe hoje e limitações

5.1 O que está paralelizado de fato

O MFSim já é paralelizado por MPI via decomposição de domínio (cada processo possui uma região física, limites $a1..b1$, etc., e patches AMR associados). No acoplamento, essa decomposição é explorada *somente* para fazer a interpolação de entrada:

- cada processo interpola MFSim→RTS apenas para pontos do RTS dentro do seu subdomínio (`in_proc`);
- o campo completo do RTS é reunido no root por `MPI_REDUCE(SUM)`;
- o RTS é resolvido no root (serial);
- os resultados são enviados a todos por `MPI_BCAST`.

Ou seja: há MPI, mas o solver radiativo não está paralelizado.



5.2 Principais gargalos observáveis no desenho atual

1. **Gargalo de tempo:** o tempo de `rtesolve` não escala (só no root).
2. **Gargalo de comunicação:** redução de campos 3D para o root e broadcast de três campos 3D a cada avaliação radiativa.
3. **Gargalo de memória:** todos os ranks alocam arrays do RTS; no caso FAM/DOM, o array de intensidade `IG` é 5D e pode dominar a memória. Mesmo que só o root use `IG` para resolver, hoje ele está alocado em todos os processos na inicialização.
4. **Custo de (re)mapear malhas:** `update_interp_map` pode ser caro quando há muitos patches AMR.

O que são Ranks?

RTS teria que usar mesmas malhas do MFSim?

5.3 Observação sobre o módulo `rad_parallel.f90`

O arquivo `src_term/RTS/rad_parallel.f90` contém rotinas como `gather_field` e `create_global_field_map` que sugerem uma tentativa (ou protótipo) de paralelização/serialização de campos do MFSim via `MPI_ALLGATHER`. No fluxo efetivo do acoplamento, entretanto, essas rotinas **não são usadas**; o caminho ativo para construir campos do RTS é o de `rad_interface.f90` com `MPI_REDUCE(SUM)`.

Além disso, há indícios de que o módulo `rad_parallel` está *incompleto ou não validado*: o tipo `rad_proc_map` declara ponteiros (`proc(:)`, `begin(:)`, `end(:)`) que precisariam ser alocados antes de uso, mas em `create_global_field_map` eles são indexados diretamente. A recomendação é tratar esse arquivo como *experimental* (ou removê-lo do caminho crítico) até que seja finalizado com testes de consistência e integração.

5.4 Notas de desempenho: chamadas globais e barreiras

A rotina `rad_main` (`rad_core.f90`) envolve chamadas globais que podem penalizar escalabilidade:

- `MPI_REDUCE` para cada campo de entrada interpolado MFSim→RTS.
- três `MPI_BCAST` para disseminar `S_rad`, `Q_radw` e `G`.
- `MPI_BARRIER` em pontos do fluxo (antes/depois de comunicação e do solver). Barreiras são úteis para depuração/temporização, mas em produção tendem a *forçar sincronização* e reduzir sobreposição comunicação-cálculo. Idealmente, deveriam ser removidas (ou condicionadas a flags de debug/profiling).

Procurar no código

5.5 Estimativas rápidas de memória para IG (impacto da paralelização)

Os métodos FAM e DOM armazenam a intensidade radiativa IG em 5D. Em dupla precisão (8 bytes por elemento), a memória aproximada é:

- **FAM:** M_{IG}
 $approx8$
 $, n_{xi}$
 $, n_{yi}$
 $, n_{zi}$
 $, n_t$
 $, n_p$ bytes.
- **DOM:** M_{IG}
 $approx8$
 $, n_{xi}$
 $, n_{yi}$
 $, n_{zi}$
 $, n_q$
 $, 8$ bytes.

Para referência, com $n_{xi} = 258$ ($nx = 256$), $n_{yi} = 258$, $n_{zi} = 258$ e $n_t n_p = 128$, o IG do FAM já entra na ordem de múltiplos **gigabytes por rank** se o domínio for replicado. Isso torna a decomposição espacial e/ou angular não apenas desejável, mas *necessária* para casos 3D grandes.

6 Estratégias de paralelização MPI para o RTS (com prós e contras)

A seguir estão opções *deliberadamente abertas* para paralelizar o RTS. Elas não são mutuamente exclusivas; várias podem ser combinadas em um desenho híbrido.

6.1 Resumo comparativo das opções

Opção	Paralelismo	Mudança no RTS	Escala típica	Observações
0	Memória (não paraleliza solver)	Pequena	Nenhuma (tempo), mas viabiliza casos maiores	Evita alocar IG (5D) em todos os ranks; mantém root-only.
1	Bandas espectrais (IBND)	Baixa-média	Até nsb ranks (ou alguns múltiplos)	Ótimo para WSGG/NG, pouca comunicação; não resolve modo cinza.
2	Ângulos/direções	Média	Até $\#$ ângulos (FAM) ou $\#$ direções (DOM)	Muito atrativa sem espalhamento; com espalhamento exige reduções globais.
3	Espaço (decomposição de domínio)	Alta	Alta (malhas grandes)	Exige wavefront (KBA) para sweeps; elimina reduce+bcast do acoplamento se alinhado ao MF-Sim.

4	Híbrido (espaço+ângulo+banda)	Muito alta	Muito alta	Melhor uso de clusters grandes, mas maior complexidade de engenharia.
5	Espaço com halos “defasados” (Jacobi)	Média	Média	Simplifica comunicação, mas piora convergência e muda o algoritmo.
6	Rays/caminhos (reformular FAM)	Muito alta	Variável	Mudança grande, pode valer para geometrias específicas.

6.2 Opção 0: reduzir consumo de memória sem paralelizar o solver

Ideia. Manter o solver no root, mas evitar alocar as grandes estruturas 5D (ex.: `IG`, `Sm`, `phase_f`) em processos não-root.

Como.

- Mover alocações pesadas para um bloco `if (proc_id==0)` dentro de `rad_start` e/ou `start_connection`.
- Nos demais ranks, alocar apenas o necessário para interpolação e recepção (`T_energy`, `G`, `S_rad`, `Q_radw`, etc.).

Prós.

- Alteração pequena, risco baixo.
- Pode ser o diferencial entre “roda” e “explode memória” em casos 3D maiores.

Contras.

- Não reduz o tempo total do solver, apenas permite casos maiores.
- Comunicação (`reduce+bcast`) permanece.

6.3 Opção 1: decomposição por bandas espectrais (MPI sobre IBND)

Ideia. Nos modos não-cinza (`nongray_flag=.true.`), a RTE é resolvida para cada banda `IBND=1..nsb` e os resultados são somados. Cada banda é independente (exceto pela soma final), então é natural paralelizar sobre bandas.

Como.

- Distribuir bandas entre ranks: cada rank resolve um subconjunto de IBND.
- Cada rank computa contribuições locais `Gband` e `Sband`; ao final, usa `MPI_ALLREDUCE(SUM)` para obter `G` e `S_rad` globais.
- Fluxos de parede: somar contribuições ou reunir `IG` de bandas só para paredes.

Prós.

- Implementação relativamente simples (alterar o `BAND_LOOP`).
- Boa eficiência: pouca comunicação (apenas somas finais por iteração ou por banda).

Contras.

- Escala limitada por **nsb** (p.ex., WSGG típico tem 5 bandas).
- Não ajuda no modo cinza.
- Ainda replica o domínio espacial em todos os ranks (a menos que combinado com outra decomposição).

6.4 Opção 2: decomposição angular (MPI sobre direções/ângulos)

Ideia. As direções discretas são um eixo natural de paralelismo:

- DOM: direções (l, m) com $l = 1..nq$ e $m = 1..8$.
- FAM: direções (θ_l, ϕ_m) com $l = 1..nt$ e $m = 1..np$.

Cada direção resolve uma equação de transporte *independente* dado o termo fonte S_m . Assim, cada rank pode cuidar de um subconjunto de ângulos.

Comunicações necessárias.

- Se $\sigma_{rad} = 0$ (sem espalhamento): quase não há acoplamento angular; basta reduzir G e fluxos de parede ao final.
- Com espalhamento: S_m depende de integrais no ângulo. Isso exige **MPI_ALLREDUCE** por célula para acumular o termo de espalhamento, ou uma estratégia de acumulação local + redução.
- Condições de contorno com reflexão (FAM/DOMbound_in) usam somas de intensidades de saída; também podem ser implementadas por redução parcial.

Prós.

- Não exige implementar sweeps distribuídos por subdomínio (evita a complexidade do wavefront de transporte).
- Pode reduzir memória por rank (cada rank armazena IG apenas para seus ângulos).
- Boa escalabilidade quando há muitos ângulos (FAM típico pode ter dezenas/centenas).

Contras.

- Ainda replica o domínio espacial (campos T_{energy} , $cappa$, etc.) em todos os ranks.
- Com espalhamento, o custo de comunicação do **ALLREDUCE** pode ficar alto (depende do tamanho do domínio).
- Necessita refatorar a forma como IG é armazenado (dimensão angular passa a ser local/particionada).

6.5 Opção 3: decomposição de domínio espacial (MPI sobre i, j, k)

Ideia. Esta é a opção mais alinhada ao pedido de *decomposição de domínio*: cada rank armazena e resolve apenas um sub-bloco da malha do RTS, com células fantasma (halos) e trocas por MPI.



3.1. Benefício adicional: acoplamento “natural” com o MFSim

Se o particionamento espacial do RTS for escolhido para coincidir com as caixas físicas do MFSim (os limites $a1..b1$, $a2..b2$, $a3..b3$ já existentes), então:

- MFSim→RTS: não é necessário MPI_REDUCE; cada rank interpola apenas sua parte do RTS e isso já é o campo correto local. 
- RTS→MFSim: não é necessário MPI_BCAST; cada rank já possui S_{rad} e G no seu subdomínio e interpola diretamente para suas células do MFSim.

3.2. O desafio: sweeps direcionais exigem wavefront (KBA)

Os métodos DOM e FAM usam sweeps *upwind* (dependência de vizinhos a montante). Em decomposição espacial, para um dado octante, um rank só pode começar a varredura quando receber as intensidades “de entrada” vindas dos ranks a montante (p.ex., faces oeste/sul/fundo para o octante $+x, +y, +z$). Isso leva ao padrão clássico de **wavefront sweep** (ex.: algoritmo KBA em transporte de nêutrons).

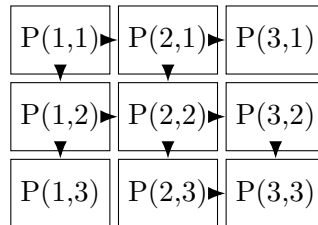


Figura 2: Ilustração 2D do wavefront: para um sweep no sentido $+x, +y$, processos em “diagonais” podem executar após receber dados dos vizinhos a montante. Em 3D, o conceito se estende a frentes 2D de processos.

3.3. Esqueleto de implementação (KBA/wavefront) para DOM/FAM

Abaixo está um esqueleto (alto nível) do que normalmente se implementa para sweeps paralelos em transporte em malha estruturada. A ideia é repetir a lógica para cada octante (combinação de sinais) e, dentro de cada octante, para cada direção (DOM) ou par (θ, ϕ) (FAM) pertencente àquele octante.

Listing 3: Pseudocódigo do sweep por decomposição espacial (wavefront/KBA).

```
! Dados:
! - MPI Cartesian coords: (px,py,pz)
! - Neighbors: west/east/south/north/bottom/top
! - Local subdomain: i=i0..i1, j=j0..j1, k=k0..k1 (com halos)
! - For each octant: upstream neighbors depend on sign(mu,eta,xi)

do iter = 1, ITMAX
  ! 1) Monta Sm local (emissao + espalhamento) para as celulas locais
  call RHS_SM_local(Sm_local, IG_local, ...)

  eps_local = 0.0d0

  do oct = 1, 8
    ! 2) Posta recepcoes (faces de entrada internas) vindas dos vizinhos a montante
    call post_Irecv_inflow_planes(oct, ...)

    ! 3) Se for fronteira fisica no octante, aplica BC de entrada nas faces
    call apply_physical_inflow_BC(oct, ...)
```

```

! 4) Espera chegar o que for necessario para iniciar
call wait_inflow_dependencies(oct, ...)

! 5) Sweep no bloco local (ordem de i,j,k depende do octante)
do each direction d in octant(oct)
  do k in sweep_order_k(oct)
    do j in sweep_order_j(oct)
      do i in sweep_order_i(oct)
        Ip = IG_local(i,j,k,d)
        ! usa vizinhos a montante (podem estar no halo)
        call rad_scheme(IG_local(i-dix,j,k,d), IG_local(i,j-diy,k,d), IG_local(i,j,k-
          diz,d), Ip, Iw, Is, Ib)
        IG_local(i,j,k,d) = update(Iw,Is,Ib,Sm_local(i,j,k,d), ...)
        eps_local = max(eps_local, abs(IG_local(i,j,k,d)-Ip)/(IG_local(i,j,k,d)+small)
      )
    end do
  end do
end do

! 6) Envia faces de saida para vizinhos a jusante
call Isend_outflow_planes(oct, ...)
end do

! 7) Convergencia global
call MPI_ALLREDUCE(eps_local, eps_global, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD, ierr)
if (eps_global < rad_tol) exit
end do

```

Tamanho de mensagens. Em decomposição espacial pura, o dado que trafega entre vizinhos são planos de intensidade (faces) por direção. Uma estimativa típica para uma mensagem de uma face é:

$$textbytes \approx 8; N_{textface}; N_{Omega, textoct},$$

onde $N_{textface}$ é o número de células no plano (ex.: n_y^{loc} , n_z^{loc}) e $N_{Omega, textoct}$ é o número de ângulos naquele octante (DOM: n_q ; FAM: cerca de $n_t n_p / 8$). Isso explica por que o híbrido (Opção 4) é atraente: dividir ângulos reduz o tamanho das mensagens.

Observação. O FAMbound_in/DOMbound_in usa integrais de saída (reflexão difusa). Em decomposição espacial, a reflexão em *paredes físicas* continua local (apenas ranks que tocam a parede), mas em *interfaces internas* a condição é simplesmente continuidade: a saída de um subdomínio é a entrada do vizinho no octante correspondente.

Prós.

- Melhor caminho para escalar para malhas grandes (memória e tempo).
- Elimina reduce+bcast do acoplamento (se alinhado ao MFSim).
- Comunicação é majoritariamente entre vizinhos (topologia de grade).

Contras.

- Maior esforço de implementação (refatoração do RTS para arrays locais e comunicação por octante/direção).

- O paralelismo é limitado pelo tamanho da frente (wavefront), especialmente em fortes escalas (muitos ranks para poucos pontos por rank).
- É preciso atenção especial para condições de contorno com reflexão e para o cálculo de resíduo global.

6.6 Opção 4: híbrido (domínio espacial + ângulos + bandas)

Ideia. Combinar decomposição espacial (para memória e escalabilidade grosseira) com decomposição angular/bandas (para reduzir comunicação por sweep e melhorar escalabilidade fina).

Desenho típico.

- Criar uma topologia MPI cartesiana para o espaço (P_x $\times$ P_y $\times$ P_z).
- Para cada subdomínio espacial, criar um subcomunicador para ângulos/bandas (por `MPI_Comm_split`).
- Dentro do subdomínio, distribuir direções e/ou bandas.
- Reduções de espalhamento e de $\mathbf{G}$ acontecem *dentro do subcomunicador* ou globalmente conforme necessidade física.



Prós.

- Melhor potencial de escalabilidade em clusters grandes.
- Reduz o volume de dados por mensagem (cada rank carrega menos ângulos).

Contras.

- Complexidade significativamente maior (comunicadores múltiplos, layout de dados).
- Mais difícil de depurar e validar.

6.7 Opção 5: “decomposição de domínio com fontes defasadas” (Jacobi) para evitar wavefront

Ideia. Uma alternativa “mais simples” à implementação KBA é quebrar a dependência direta de montante usando uma forma mais Jacobi: em cada iteração, usar intensidades de contorno internas (entre subdomínios) *da iteração anterior* como condição de entrada, e só depois trocar halos.

Prós.

- Implementação mais simples: comunicação por troca de halos padrão.
- Fácil de acoplar com malhas decompostas sem wavefront.

Contras.

- Convergência pode piorar bastante (mais iterações) e/ou exigir relaxação.
- Dependendo do regime óptico, pode ser instável ou muito lento.
- É uma mudança *algorítmica* (não é apenas paralelização); precisa validação física cuidadosa.

6.8 Opção 6: reinterpretação do FAM como “feixes/rays” e paralelizar por caminhos

Ideia. O FAM também pode ser implementado como um conjunto de trajetórias (características) para cada direção e célula (“long characteristics” / ray tracing). Uma estratégia MPI possível é distribuir *raios* (ou blocos de raios) entre ranks.

Prós.

- Pode reduzir dependências de sweep local, dependendo do esquema.
- Pode ser vantajoso em geometrias onde o custo está no traçado de raios.

Contras.

- Exige reescrever o solver (mudança grande).
- Comunicação e acúmulo de contribuições podem ser complexos; risco de desbalanceamento.

6.9 Opção 7: reduzir replicação usando MPI Shared Memory (por nó) ou MPI RMA

Ideia. Se, por algum motivo, for desejável manter o domínio espacial replicado (ou o solver root-only) mas reduzir o consumo de memória *por nó*, uma alternativa é alocar grandes arrays (ex.: IG) em memória compartilhada entre ranks do mesmo nó usando `MPI_Win_allocate_shared`. Assim, múltiplos ranks no mesmo nó enxergam o mesmo buffer físico.

Prós.

- Pode reduzir drasticamente a memória total consumida (por nó) sem reescrever o solver para decomposição espacial.
- Útil como medida de transição: permite testar malhas maiores e perfilar hotspots antes de uma refatoração KBA completa.

Contras.

- Não fornece speedup automático; é uma técnica de *memória*, não de paralelismo computacional.
- Exige cuidado com concorrência (se mais de um rank escrever no mesmo array) e com afinidade NUMA.
- Não resolve o gargalo de comunicação global (reduce+bcast) nem o gargalo de tempo (root-only), a menos que combinada com outra opção.

7 Recomendação prática: caminho incremental

Uma estratégia robusta costuma ser incremental:

1. **Curto prazo (baixo risco):** implementar a Opção 0 (economia de memória) e otimizar `update_interp_map` por interseção de índices.
2. **Médio prazo:** implementar Opção 1 (bandas) e/ou Opção 2 (ângulos) para obter *speedup* sem reestruturar o domínio.

3. **Longo prazo (decomposição de domínio real):** implementar Opção 3 (KBA/wavefront) e, se necessário, evoluir para Opção 4 (híbrido) para escalar a malhas grandes.

Se o objetivo principal for “decomposição de domínio” e bom casamento com o MFSim, a Opção 3 é a mais coerente: ela elimina as operações globais (reduce+bcast) e aproveita que MFSim já está decomposto espacialmente.

8 Checklist de alterações no código para uma versão RTS paralela por domínio

Esta seção lista mudanças concretas (nível “engenharia”) necessárias para uma paralelização espacial.

8.1 1) Definir subdomínio RTS por processo

- Construir uma topologia cartesiana MPI coerente com `npx, npy, npz` do MFSim.
- Calcular intervalos de índices locais ($i_0..i_1$) a partir de limites físicos (`a1..b1`) e do espaçamento do RTS ($dx = (gb1-ga1)/nx$), garantindo consistência nos pontos de borda.

8.2 2) Refatorar arrays globais do RTS para arrays locais

Exemplos:

- `T_energy`, `cappa`, `sigma_rad`, `beta`, `G`, `S_rad`, `Q_radw` passam a existir como campos locais com halo.
- IG (5D) deve ser particionado ao menos no espaço, e idealmente também no ângulo (para reduzir memória e comunicação).

8.3 3) Implementar comunicação de intensidades nos sweeps

- Para cada octante, definir quais faces são *entrada* e quais são *saída*.
- Antes do sweep: `MPI_Irecv` das faces de entrada vindas dos vizinhos a montante; ranks de fronteira física usam condição de contorno em vez de receber.
- Após computar o bloco: `MPI_Isend` das faces de saída para vizinhos a jusante.

8.4 4) Resíduo e convergência global

- Cada rank calcula seu resíduo local (`eps_local`).
- `MPI_ALLREDUCE(MAX)` obtém `eps_global` para critério de parada.

8.5 5) Acoplamento com MFSim sem operações globais

- Trocar `MPI_REDUCE` por interpolação diretamente no subdomínio local do RTS.
- Remover `MPI_BCAST` e interpolar `S_rad/G/Q_radw` localmente para o MFSim.

9 Conclusões

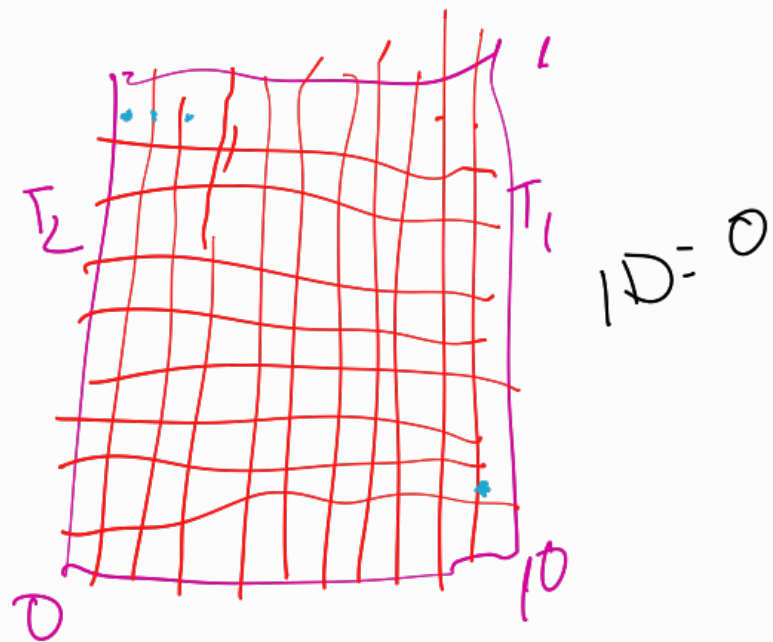
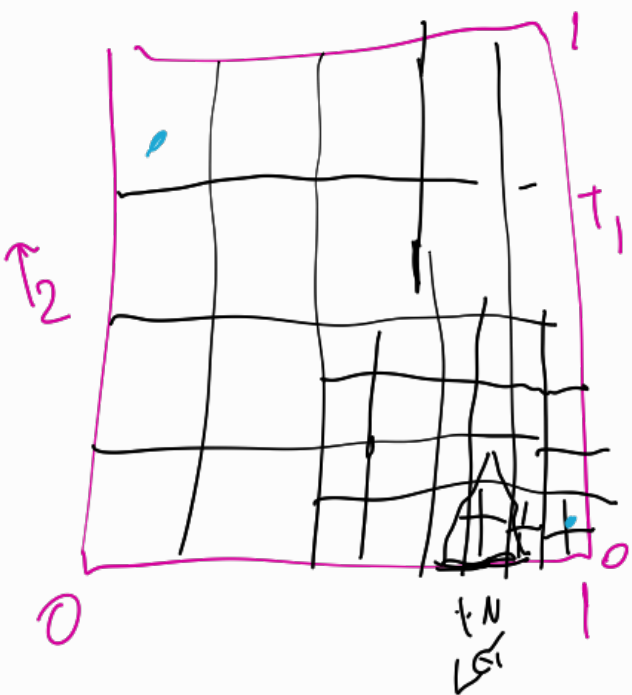
- O acoplamento MFSim-RTS atual é funcional e já explora a decomposição de domínio do MFSim para *montar* os campos de entrada radiativos, mas o **solver RTS ainda é serial** (root-only).
- Para casos moderados, o desenho atual pode ser aceitável; para casos grandes, ele encontra limites de **memória** (arrays 5D replicados) e de **tempo** (root-only).
- Existem múltiplos eixos de paralelismo naturais (bandas, ângulos, espaço). A escolha depende do regime (cinza vs não-cinza, com ou sem espalhamento), do tamanho de malha e do esforço de refatoração aceitável.
- Se o objetivo é **decomposição de domínio** com MPI (pedido explícito), o caminho mais coerente é refatorar o RTS para malha local + comunicação de sweeps (wavefront/KBA), o que também simplifica o acoplamento e remove reduções/broadcasts globais.

$$\frac{\partial p_{GT}}{\partial t} - \frac{\partial p_{C_{\text{net}} T}}{\partial x} = \frac{2}{\partial x} \left(\begin{matrix} K \partial T \\ c x \end{matrix} \right) + Q_{\text{rad}}$$



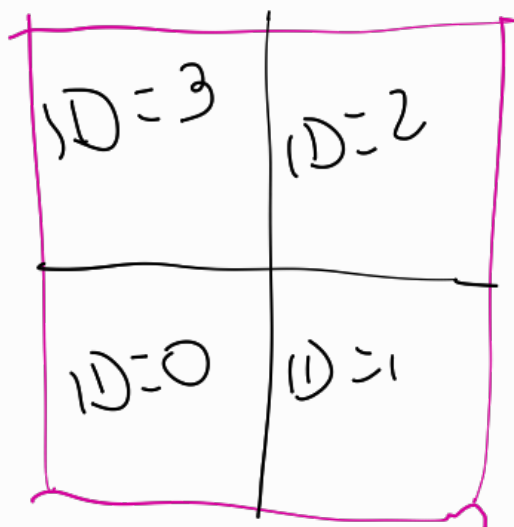
MFSim

RTS



$ID = 0$

Proc_id



t_N
 t_{N-1}
 x, y } Q_{ad}

CFD RTS

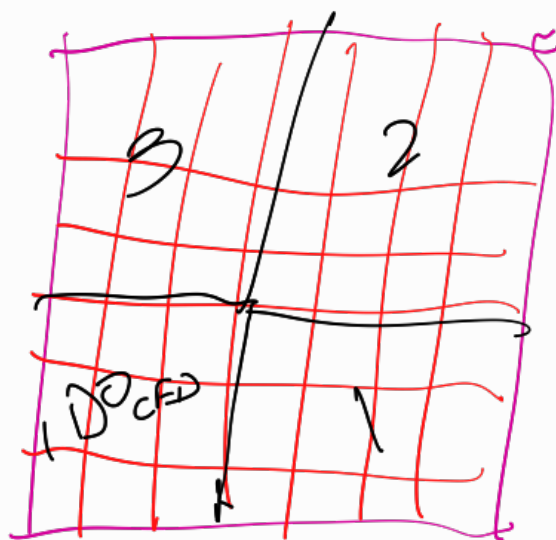
CPU₃

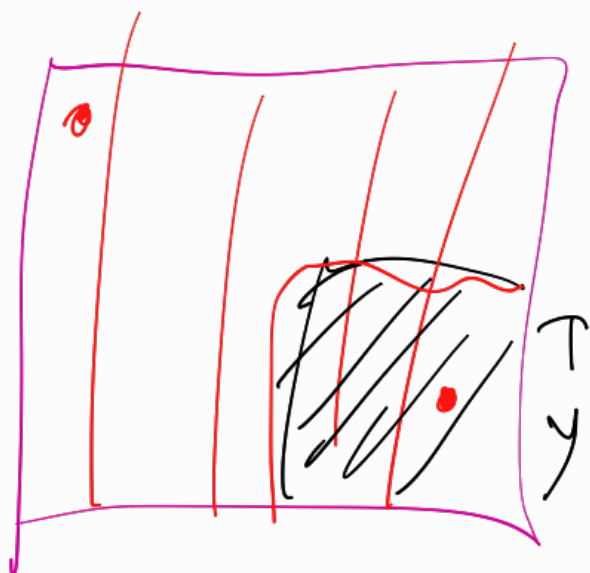
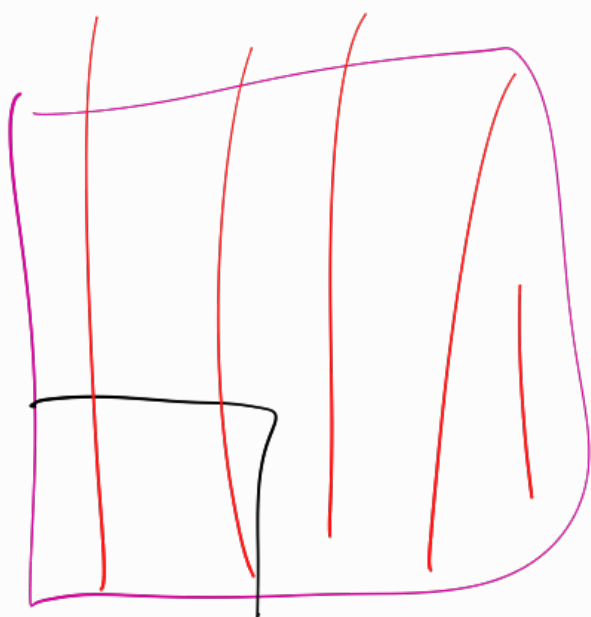
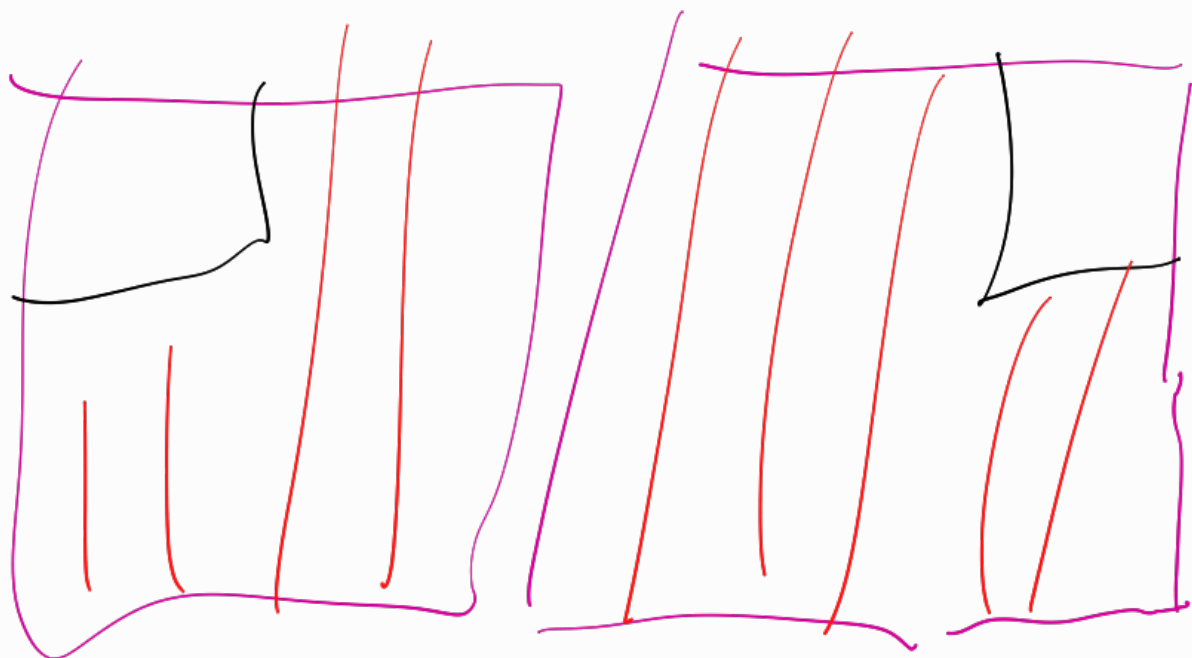
CPU₂
 0,5

CPU₀

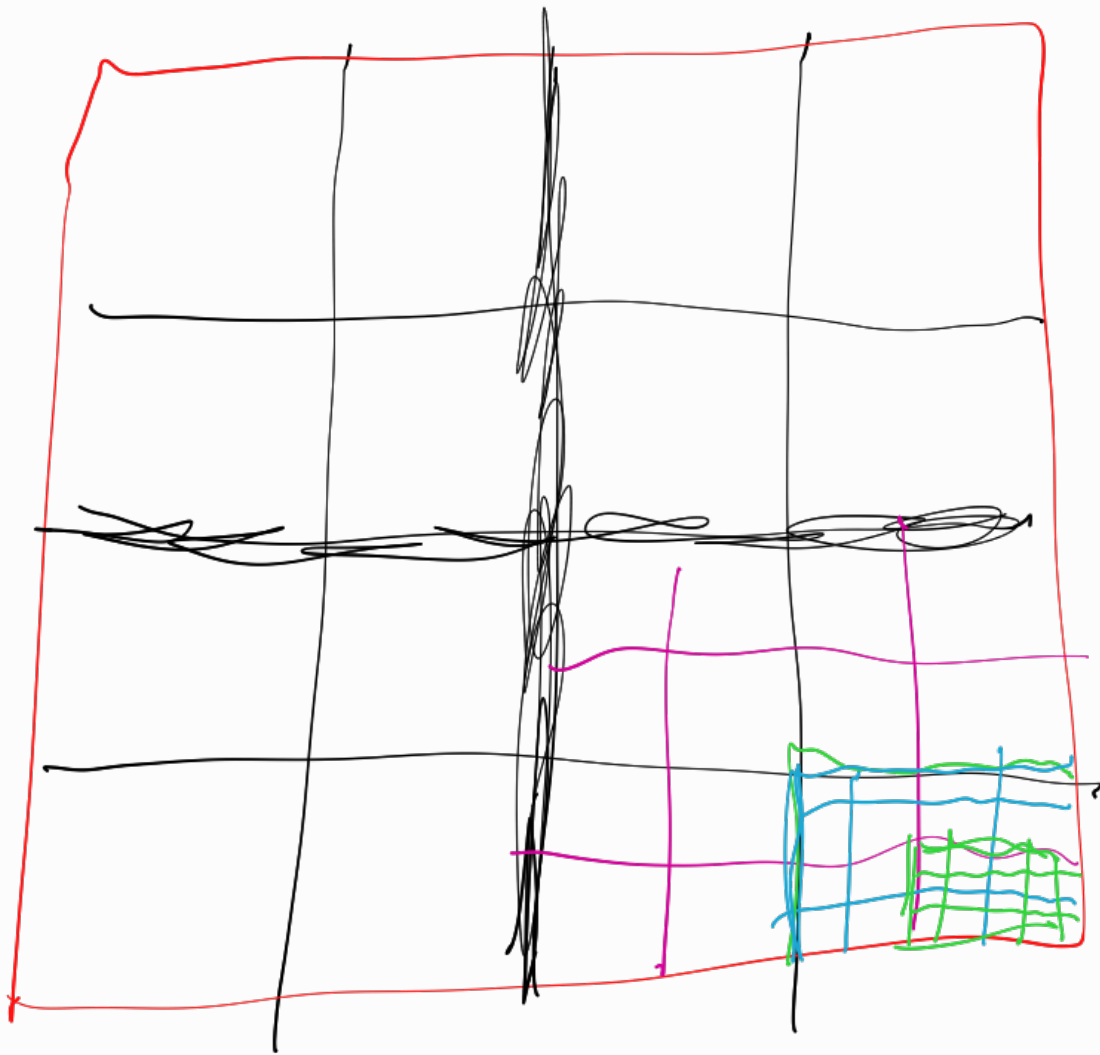
CPU₁
 0,5
 0

S | M)





Q



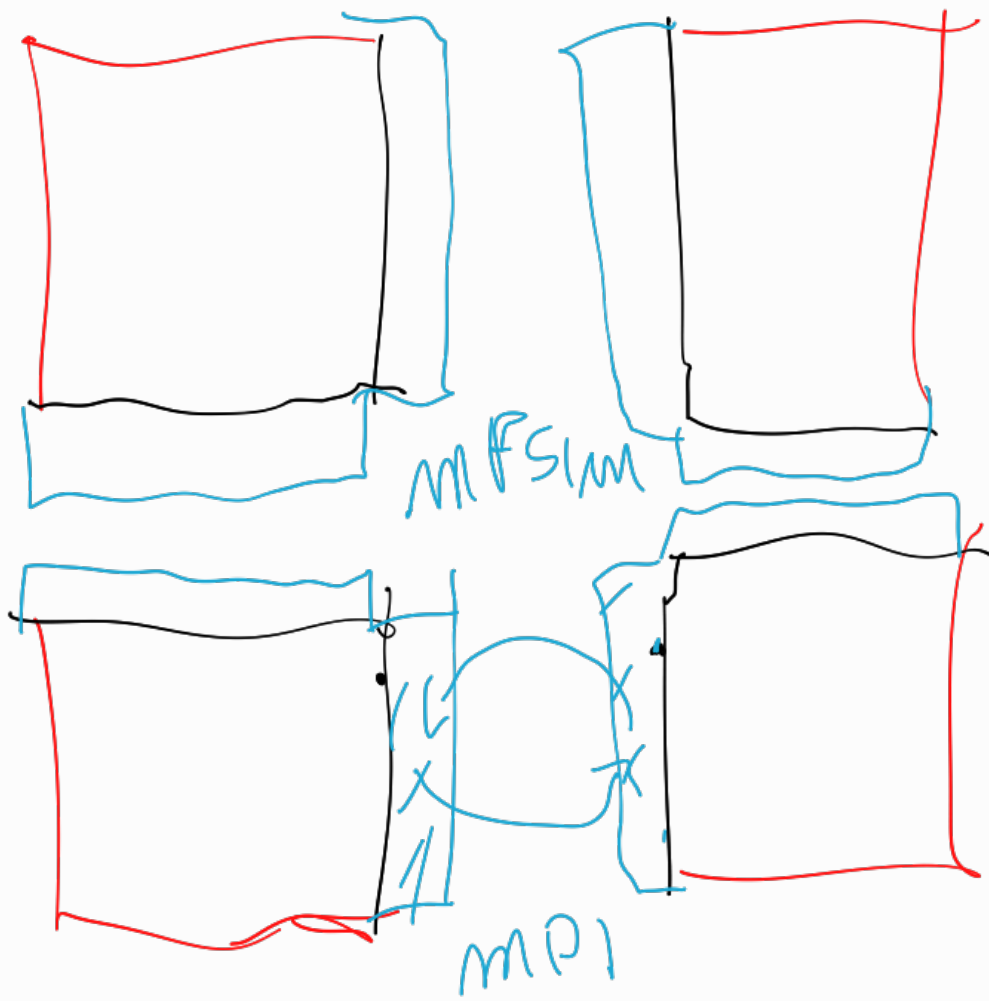
• L_{BOT}

• $L_{BOT} + 1$

• $L_{BOT} + 2$

• $L_{BOT} + 3 = L_{TOP}$

PDE



$$\frac{\partial \phi}{\partial x} \sim \frac{\phi_{i+1} - \phi_i}{\Delta x}$$

↓

The diagram shows three circles connected by a horizontal line. A blue arrow points down to the first circle on the left.